Technische Universität Braunschweig

Chair for Chip Design for Embedded Computing

HWSW-Systeme

VHDL-Übung 06: Asserts und Verification

M.Sc. Eike Trumann, M.Sc. Jasper Homann

Prof. Dr.-Ing. Guillermo Payá Vayá

9. Dezember 2025

Aufgabe VHDL-600: *Rekursiver Komperator*

Es soll eine generische Komperatorschaltung mithilfe einer rekursiven Funktion implementiert werden.

Diese nimmt zwei `std_ulogic_vector` entgegen und vergleicht diese Bitweise. Sind alle Bits gleich soll `'1'`, sonst `'0'` zurückgegeben werden. Ein Aufruf der Funktion mit zwei unterschiedlich langen Vektoren soll immer `'0'` zurückgeben.

VHDL-600

```
package recursive_compare_pkg is
  function COMPARE_FUNC (A : std_ulogic_vector; B : std_ulogic_vector) return std_ulogic;
end package;

package body recursive_compare_pkg is

    function COMPARE_FUNC (A : std_ulogic_vector; B : std_ulogic_vector) return std_ulogic is
        -- TODO: Declare variables
    begin
        -- Vectors of different length are not equal
        if A'length /= B'length then
            return '0';
        end if;

        -- Vectors of length 0 are always equal
        if A'length = 0 then
            return '1';
        end if;
```

```
        -- TODO: Handle base case (length = 1)

        -- TODO: Split input vectors and call recursively
    end function;

end package body;
```

Aufgabe VHDL-601: *Schiebeeinheiten überprüfen*

Aus den vorherigen Übungen (VHDL-402) ist die folgende Verschiebeeinheit bekannt:

```
entity best_calculator is
port(
  data_bus     : in  std_ulogic_vector(7 downto 0);
  amount       : in  std_ulogic_vector(2 downto 0);
  logic_shift  : in  std_ulogic;
  left_shift   : in  std_ulogic;
  result       : out std_ulogic_vector(7 downto 0)
);
end best_calculator;
```

Offensichtlich kann bei der Implementierung von dieser Verschiebeeinheit eine Menge schief gehen. Es ist daher notwendig, eine umfangreiche Testbench zu haben, die Fehler in der Implementierung aufspürt und möglichst hilfreich bei der Behebung der Probleme ist.

Die Aufgabe ist es, eine möglichst umfangreiche Testbench zu erstellen, die in der Lage ist, fehlerhafte Implementierungen zu finden.

```
    library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity testbench_600 is
end testbench_600;

architecture test600_arch of testbench_600 is
    component best_calculator is
        port(
-- TODO Port
        );
    end component  best_calculator;
-- TODO Signale
begin
        best_calculator_inst : best_calculator
        port map(
-- TODO Port
        );
    test : process
    begin
    -- Die Testbench muss asserts verwenden, um Testergebnisse zu melden.
```

```
    -- TODO: Tests Implementieren

    -- Der Test muss mit wait (oder std.env.stop) beendet werden.
    end process;
end architecture test600_arch;
```

Damit die CI korrekt zuordnen kann, ob eine Testbench Fehler erkannt hat, müssen asserts verwendet werden.

Testet zunächst mit der Testbench eure Implementierung aus VHDL-402.

Wenn ihr die Testbench im Git-Repository hochladet, werden unterschiedliche, teilweise fehlerhafte Implementierungen der Verschiebeeinheit mit der Testbench getestet. Eine korrekte Testbench sollte in der Lage sein, die fehlerhaften und die korrekten Implementierungen sicher voneinander zu unterscheiden. Die CI-Pipeline wird in dem Fall grün werden, dass ihr alle fehlerhaften Implementierungen korrekt erkennt.

Hinweis: Da die CI-Pipeline nur überprüft, ob eure Testbench ein fehlschlagendes assert beinhaltet, sind auch bei fehlerhaften Testbenches mehrere der Teilpipelines grün.

Aufgabe VHDL-602: *Mustererkennung überprüfen*

Aus der vorherigen Übung (VHDL-403) ist die folgende Mustererkennung bekannt:

```
entity pattern_matcher is
port(
    data_bus     : in  std_ulogic_vector(255 downto 0);
    pattern      : in  std_ulogic_vector(2 downto 0);
    counter      : out std_ulogic_vector(7 downto 0)
);
end pattern_matcher;
```

Wie immer kann bei der Implementierung dieser Mustererkennung eine Menge schief gehen. Wir haben acht verschiedene Implementierungen der Mustererkennung vorbereitet, von denen einige fehlerhaft sind. Die zu erstellende Testbench soll in der Lage sein, die fehlerhaften Implementierungen zu erkennen und möglichst hilfreiche Fehlermeldungen auszugeben.

```
-- runtime: 6 ns
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity testbench_602 is
end testbench_602;

architecture arch of testbench_602 is
    component pattern_matcher is
        port(
            -- TODO Port
        );
    end component pattern_matcher;
    -- TODO Signale
```

```
begin
        match_inst : pattern_matcher
        port map(
            -- TODO Port
        );
    test : process
    begin
        -- TODO Tests

        -- Muss mit wait oder std.env.stop beendet werden
        wait;
    end process;
end architecture arch;
```

Damit wir korrekt zuordnen können, ob eine Testbench fehler erkannt hat, müssen asserts verwendet werden. Sobald ein assert fehlschlägt, wird der Durchlauf der Testbench als nicht bestanden gewertet.

Testet zunächst mit der Testbench eure Implementierung aus VHDL-403.